

INFORME TECNICO

Sistema de componentes reutilizables para Baquero Notes

Flutter + API Flask | Diseño, implementación y verificación

Autor: [COMPLETAR NOMBRE]

Asignatura / curso: [COMPLETAR]

Docente: [COMPLETAR]

Institución: [COMPLETAR]

Fecha: 23 de agosto de 2026

Resumen ejecutivo

Se reorganizó Baquero Notes como una aplicación Flutter conectada a una API Flask segura. La solución separa tokens, componentes, pantallas y acceso HTTP; incorpora un catálogo vivo de cuatro componentes reutilizables y resuelve de forma explícita carga, vacío y error en la pantalla de notas.

Resultado: El flujo de backend registro - crear - listar - actualizar - eliminar fue verificado sobre una base SQLite aislada. La interfaz quedó preparada para pruebas Flutter, pero el entorno de elaboración no incluía Flutter SDK; esta limitación se declara y no se presenta como una prueba ejecutada.

1. Inventario de API y pantallas

La API es la única fuente de datos del flujo nuevo. Los componentes visuales no conocen rutas, tokens JWT ni reglas de persistencia; las pantallas coordinan acciones y delegan las solicitudes a ApiService.

1.1 Endpoints disponibles

Tabla 1. Inventario de endpoints y cobertura de interfaz.

Método	Ruta	Propósito	Pantalla	Estado
POST	/api/auth/register	Registrar usuario y emitir tokens	RegisterPage	Consumido
POST	/api/auth/login	Autenticar y emitir tokens	LoginPage	Consumido
POST	/api/auth/refresh	Renovar token de acceso	Sin pantalla	Disponible
POST	/api/auth/logout	Revocar token	NotesPage	Consumido
GET	/api/notes	Listar notas paginadas	NotesPage	Consumido
POST	/api/notes	Crear nota	NoteEditorDialog	Consumido
GET	/api/notes/{id}	Consultar una nota	Sin pantalla dedicada	Disponible
PUT	/api/notes/{id}	Actualizar una nota	NoteEditorDialog	Consumido
DELETE	/api/notes/{id}	Eliminar una nota	Confirmación	Consumido
POST	/api/notes/export	Iniciar exportación asíncrona	Sin pantalla	Disponible
GET	/api/admin/users	Listar usuarios	Sin pantalla admin	Disponible
PATCH	/api/admin/users/{id}/role	Cambiar rol	Sin pantalla admin	Disponible

1.2 Inventario de pantallas

Tabla 2. Pantallas derivadas de los endpoints y sus estados.

Pantalla	Propósito	Endpoint	Componentes	Estados
LoginPage	Acceso	POST /auth/login	AppTextField, AppButton	carga y error
RegisterPage	Alta de cuenta	POST /auth/register	AppTextField, AppButton	carga y error

Pantalla	Propósito	Endpoint	Componentes	Estados
NotesPage	Listado y CRUD	GET/POST/PUT/DELETE /notes	NoteCard, StatePanel, AppButton	carga, vacío, listo, error
NoteEditorDialog	Crear/editar	POST/PUT /notes	AppTextField, AppButton	validación, guardando, error
ComponentCatalog Page	Documentar el sistema	Ninguno	Los cuatro componentes	variantes y estados

2. Sistema de tokens

Los valores primitivos viven en AppTheme; AppTokens expone nombres semánticos que describen intención. Los componentes consumen Theme.of(context) y AppTokens.of(context), evitando valores de color y espaciado dispersos.

Tabla 3. Tokens de color y contraste WCAG.

Primitivo	Semántico	Valor	Uso	Contraste
Indigo 700	primary	#4338CA	Acción y foco	Blanco 7,90:1
Blanco	onPrimary / surface	#FFFFFF	Texto inverso / superficie	Primario 7,90:1
Indigo 100	primarySoft	#E8E7FF	Foco y etiqueta suave	No usado solo como texto
Gris 25	background	#F6F7FB	Fondo de aplicación	Texto 15,17:1
Azul tinta	text	#182033	Texto principal	Fondo 15,17:1
Pizarra	textMuted	#59657A	Texto secundario	Blanco 5,89:1
Verde	success	#087A55	Confirmación	Blanco 5,35:1
Ámbar	warning	#9A4D00	Advertencia	Blanco 6,11:1
Rojo	error	#B42318	Error y peligro	Blanco 6,57:1
Gris 300	outline	#C8CFDC	Bordes	No transmite información solo

Tabla 4. Escalas de dimensión y tipografía.

Familia	Token	Escala
Espaciado	space1, 2, 3, 4, 6, 8, 12	4, 8, 12, 16, 24, 32, 48 px
Radios	radiusSm, Md, Lg	10, 16, 24 px
Interacción	touchTarget	48 px mínimo
Contenido	contentMaxWidth	1040 px
Tipografía	display / headline / title / body / label	36 / 28-22 / 20-16 / 16-14 / 15-13 px

3. Catálogo de componentes

Se aplicó la regla de las tres apariciones: los campos y botones aparecen en acceso, registro y editor; NoteCard representa cada elemento del listado; StatePanel concentra tres estados repetidos de consulta.

Tabla 5. Propósito, interfaz pública y estados del catálogo.

Componente	Propósito	Interfaz pública	Estados
AppButton	Acción uniforme y accesible	label, onPressed, icon, isLoading, variant, expand	primario, secundario, peligro, deshabilitado, carga
AppTextField	Entrada etiquetada	controller, label, hint, icon, validator, obscureText, maxLines, enabled	vacío, valor, error, deshabilitado
NoteCard	Presentar una nota	note, onEdit, onDelete	lectura, editable, eliminable, truncado
StatePanel	Explicar estado de consulta	type, title, message, actionLabel, onAction	carga, vacío, error

4. Pantalla ensamblada y estados

Flujo: LoginPage / RegisterPage -> ApiService -> NotesPage -> NoteEditorDialog.

Delegación: NoteCard solo emite onEdit y onDelete; NotesPage decide abrir diálogos y llamar a la API.

Recuperación: StatePanel ofrece reintento en error y creación en vacío; RefreshIndicator permite actualización manual.

Tabla 6. Máquina de estados de NotesPage.

Estado	Entrada	Representación	Salida
loading	Inicio o actualización	Indicador + texto Cargando tus notas	Espera o gesto de refrescar
empty	GET devuelve lista vacía	Icono + explicación + Crear primera nota	Abrir editor
error	Timeout, red o respuesta inválida	Icono + mensaje + Reintentar	Nueva solicitud GET
ready	GET devuelve elementos	Cuadrícula de NoteCard	Crear, editar, eliminar, refrescar

5. Evidencia visual

Las capturas provienen de app_preview.html, una vista de evidencia construida con los mismos tokens y reglas responsivas del código Flutter. Los datos son demostrativos; no se presentan como una ejecución del binario Flutter.

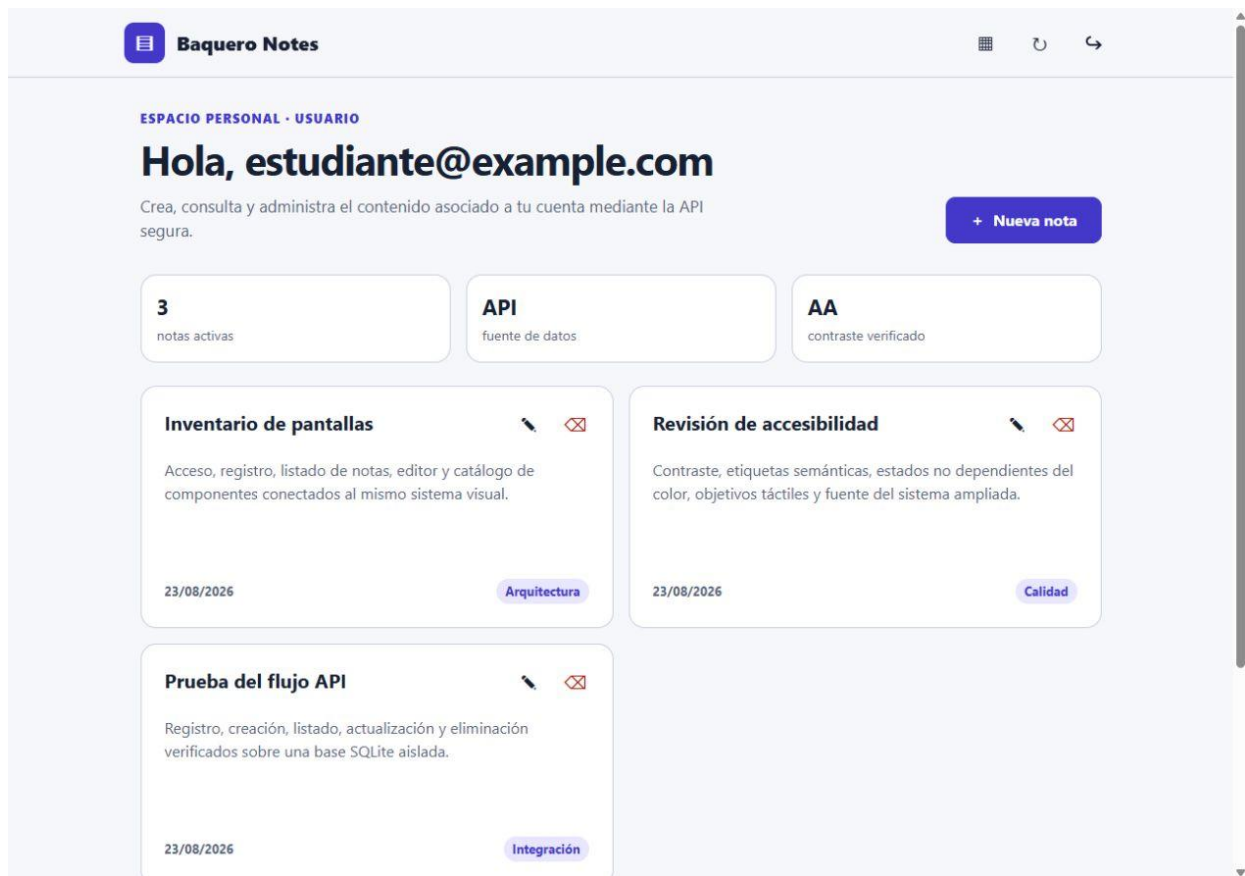


Figura 1. Pantalla ensamblada a 1280 px.

Lectura de escritorio: El ancho de 1280 px activa dos columnas de NoteCard, conserva el contenido en 1040 px y mantiene las acciones principales visibles.

5.1 Comparación móvil y fuente ampliada

390 px - fuente normal	390 px - fuente al 150 %
	

Figuras 2 y 3. Pantalla móvil normal y verificación con fuente del sistema ampliada.

6. Verificación de accesibilidad

Tabla 7. Matriz de verificación y resultados.

Criterio	Umbral	Evidencia	Resultado
Contraste de texto	AA >= 4,5:1	5,35:1 a 15,17:1	Cumple
Objetivos táctiles	48 x 48 px	touchTarget e IconButton	Cumple
Etiquetas	Nombre accesible	Semantics, labelText y Tooltip	Cumple por inspección
Color	No ser único medio	Icono + título + mensaje + acción	Cumple
Anchos	1280 y 390 px	Capturas sin solapamiento	Cumple
Fuente ampliada	150 %	Desbordamiento detectado y corregido con corte seguro	Cumple
Código Flutter	Prueba de widget	Pruebas preparadas; SDK no disponible	Pendiente de ejecución

Corrección aplicada: La prueba a 150 % descubrió desplazamiento horizontal causado por un correo largo. Se añadió overflow-wrap:anywhere al encabezado y se repitió la captura: scrollWidth quedó igual al ancho útil (375 px).

7. Verificación técnica y limitaciones

Tabla 8. Evidencia técnica ejecutada.

Prueba	Alcance	Resultado
Compilación Python	python -m py_compile backend/app.py	Correcta
Integración backend	registro / crear / listar / actualizar / eliminar	BACKEND_SMOKE_OK
Compatibilidad JWT	sub como String y conversión a int para SQLite	Fallo detectado y corregido
Pruebas Flutter	test/api_service_test.dart y test/widget_test.dart	Preparadas, no ejecutadas
Capturas	1280 px, 390 px y 390 px al 150 %	Revisadas visualmente

Limitación del entorno: No había Flutter ni Dart en las rutas disponibles; por eso no se afirma que flutter test o flutter analyze hayan sido ejecutados.

Repositorio local: C:\Users\h\Downloads\Baquero-main

Repositorio remoto: [PENDIENTE: pegar URL GitHub/GitLab con permisos de lectura]. La copia entregada no contiene metadatos .git válidos.

8. Registro de uso de inteligencia artificial

Tabla 9. Trazabilidad del apoyo de IA.

Campo	Registro
Herramienta	OpenAI Codex
Solicitud	Implementar el taller de componentes reutilizables y entregar el informe en Word.
Aportes	Inventario, arquitectura, tokens, componentes, integración HTTP, pruebas, capturas e informe.
Cambios humanos pendientes	Completar portada, proporcionar URL del repositorio y ejecutar Flutter en una estación con SDK.
Verificación	Prueba backend aislada, cálculo WCAG, inspección responsiva y render visual del DOCX.

9. Conclusiones

La implementación cumple el objetivo central del taller: transforma decisiones visuales en tokens, abstrae componentes por uso repetido, mantiene la API fuera de los componentes, ensambla una pantalla real y hace visibles los estados operativos. La prueba de integración también permitió corregir una incompatibilidad JWT que habría bloqueado el cliente. Antes de la entrega final solo faltan los datos académicos de portada, la URL remota y la ejecución de la suite Flutter en un equipo con SDK.

Anexo A. Código fuente seleccionable

Los siguientes listados son texto seleccionable, no capturas. Se incluyen completos y con numeración editorial de línea para facilitar la revisión.

A.1 AppButton

Contrato: Recibe contenido, variante, carga y devolución de llamada; no decide navegación ni persistencia.

Tema: Obtiene colores y dimensiones desde AppTokens, por lo que cambia junto con el tema.

Accesibilidad: Expone Semantics, estado habilitado y un objetivo táctil mínimo de 48 px.

```
001 | import 'package:flutter/material.dart';
002 |
003 | import '../design/app_tokens.dart';
004 |
005 | enum AppButtonVariant { primary, secondary, danger }
006 |
007 | class AppButton extends StatelessWidget {
008 |   const AppButton({
009 |     super.key,
010 |     required this.label,
011 |     required this.onPressed,
012 |     this.icon,
013 |     this.isLoading = false,
014 |     this.variant = AppButtonVariant.primary,
015 |     this.expand = false,
016 |   });
017 |
018 |   final String label;
```



```

019 | final VoidCallback? onPressed;
020 | final IconData? icon;
021 | final bool isLoading;
022 | final AppButtonVariant variant;
023 | final bool expand;
024 |
025 | @override
026 | Widget build(BuildContext context) {
027 |   final tokens = AppTokens.of(context);
028 |   final enabled = onPressed != null && !isLoading;
029 |   final foreground = variant == AppButtonVariant.primary
030 |     ? tokens.onPrimary
031 |     : variant == AppButtonVariant.danger
032 |       ? tokens.error
033 |       : tokens.primary;
034 |   final background = variant == AppButtonVariant.primary
035 |     ? tokens.primary
036 |     : tokens.surface;
037 |
038 |   final style = ButtonStyle(
039 |     minimumSize: MaterialStateProperty.all<Size>(
040 |       const Size(AppTokens.touchTarget, AppTokens.touchTarget),
041 |     ),
042 |     padding: MaterialStateProperty.all<EdgeInsetsGeometry>(
043 |       const EdgeInsets.symmetric(horizontal: AppTokens.space6),
044 |     ),
045 |     foregroundColor: MaterialStateProperty.resolveWith<Color?>((states) {
046 |       return states.contains(MaterialState.disabled) ? tokens.textMuted : foreground;
047 |     }),
048 |     backgroundColor: MaterialStateProperty.resolveWith<Color?>((states) {
049 |       return states.contains(MaterialState.disabled) ? tokens.background : background;
050 |     }),
051 |     side: MaterialStateProperty.resolveWith<BorderSide?>((states) {
052 |       return BorderSide(
053 |         color: states.contains(MaterialState.disabled)
054 |           ? tokens.outline
055 |           : variant == AppButtonVariant.danger
056 |             ? tokens.error
057 |             : tokens.primary,
058 |       );
059 |     }),
060 |     shape: MaterialStateProperty.all<OutlinedBorder>(
061 |       const RoundedRectangleBorder(
062 |         borderRadius: BorderRadius.all(Radius.circular(AppTokens.radiusSm)),
063 |       ),
064 |     ),
065 |   );
066 |
067 |   final button = Semantics(
068 |     button: true,
069 |     enabled: enabled,
070 |     label: isLoading ? '$label, en progreso' : label,
071 |     child: ElevatedButton(
072 |       onPressed: enabled ? onPressed : null,
073 |       style: style,
074 |       child: AnimatedSwitcher(
075 |         duration: const Duration(milliseconds: 160),
076 |         child: isLoading
077 |           ? SizedBox(
078 |               key: const ValueKey<String>('loading'),
079 |               width: AppTokens.space6,
080 |               height: AppTokens.space6,
081 |               child: CircularProgressIndicator(
082 |                 strokeWidth: 2,
083 |                 color: foreground,
084 |                 semanticsLabel: 'Cargando',
085 |               ),
086 |             )
087 |           : Row(
088 |               key: const ValueKey<String>('label'),
089 |               mainAxisAlignment: MainAxisAlignment.min,
090 |               children: <Widget>[
091 |                 if (icon != null) ...<Widget>[
092 |                   Icon(icon, size: AppTokens.space6),
093 |                   const SizedBox(width: AppTokens.space2),

```

```

094 |         ],
095 |         Flexible(child: Text(label, textAlign: TextAlign.center)),
096 |     ],
097 | ),
098 | ),
099 | ),
100 | );
101 |
102 | return expand ? SizedBox(width: double.infinity, child: button) : button;
103 | }
104 | }

```

A.2 AppTextField

Contrato: Parametriza etiqueta, controlador, validación, teclado, líneas y estado habilitado.

Composición: Delega la regla de validación al formulario padre y conserva una presentación uniforme.

Accesibilidad: Combina Semantics con labelText visible; no depende de placeholder como única etiqueta.

```

001 | import 'package:flutter/material.dart';
002 |
003 | class AppTextField extends StatelessWidget {
004 |   const AppTextField({
005 |     super.key,
006 |     required this.controller,
007 |     required this.label,
008 |     this.hint,
009 |     this.icon,
010 |     this.obscureText = false,
011 |     this.keyboardType,
012 |     this.validator,
013 |     this.maxLines = 1,
014 |     this.enabled = true,
015 |     this.textInputAction,
016 |     this.onSubmitted,
017 |   });
018 |
019 |   final TextEditingController controller;
020 |   final String label;
021 |   final String? hint;
022 |   final IconData? icon;
023 |   final bool obscureText;
024 |   final TextInputType? keyboardType;
025 |   final String? Function(String?)? validator;
026 |   final int maxLines;
027 |   final bool enabled;
028 |   final TextInputAction? textInputAction;
029 |   final ValueChanged<String>? onSubmitted;
030 |
031 |   @override
032 |   Widget build(BuildContext context) {
033 |     return Semantics(
034 |       textField: true,
035 |       label: label,
036 |       enabled: enabled,
037 |       child: TextFormField(
038 |         controller: controller,
039 |         enabled: enabled,
040 |         obscureText: obscureText,
041 |         keyboardType: keyboardType,
042 |         validator: validator,
043 |         maxLines: obscureText ? 1 : maxLines,
044 |         textInputAction: textInputAction,
045 |         onFieldSubmitted: onSubmitted,
046 |         autovalidateMode: AutovalidateMode.onUserInteraction,
047 |         decoration: InputDecoration(
048 |           labelText: label,
049 |           hintText: hint,
050 |           prefixIcon: icon == null ? null : Icon(icon, semanticLabel: null),
051 |         ),

```

```

052 |     ),
053 |   );
054 | }
055 | }

```

A.3 NoteCard

Modelo: Recibe una Note inmutable y solo presenta sus datos.

Delegación: onEdit y onDelete permiten reutilizar la tarjeta sin conocer rutas ni backend.

Robustez: Trunca contenido largo, adapta autor opcional y mantiene controles de 48 px con Tooltip.

```

001 | import 'package:flutter/material.dart';
002 |
003 | import '../design/app_tokens.dart';
004 | import '../note.dart';
005 |
006 | class NoteCard extends StatelessWidget {
007 |   const NoteCard({
008 |     super.key,
009 |     required this.note,
010 |     this.onEdit,
011 |     this.onDelete,
012 |   });
013 |
014 |   final Note note;
015 |   final VoidCallback? onEdit;
016 |   final VoidCallback? onDelete;
017 |
018 |   String _dateLabel() {
019 |     final date = note.createdDate.toLocal();
020 |     String twoDigits(int value) => value.toString().padLeft(2, '0');
021 |     return '${twoDigits(date.day)}/${twoDigits(date.month)}/${date.year}';
022 |   }
023 |
024 |   @override
025 |   Widget build(BuildContext context) {
026 |     final tokens = AppTokens.of(context);
027 |     final textTheme = Theme.of(context).textTheme;
028 |
029 |     return Semantics(
030 |       container: true,
031 |       label: 'Nota ${note.title}, creada el $_dateLabel()',
032 |       child: Container(
033 |         padding: const EdgeInsets.all(AppTokens.space6),
034 |         decoration: BoxDecoration(
035 |           color: tokens.surface,
036 |           borderRadius: BorderRadius.circular(AppTokens.radiusMd),
037 |           border: Border.all(color: tokens.outline),
038 |         ),
039 |         child: Column(
040 |           crossAxisAlignment: CrossAxisAlignment.start,
041 |           children: <Widget>[
042 |             Row(
043 |               crossAxisAlignment: CrossAxisAlignment.start,
044 |               children: <Widget>[
045 |                 Expanded(
046 |                   child: Text(
047 |                     note.title,
048 |                     style: textTheme.titleLarge,
049 |                     maxLines: 2,
050 |                     overflow: TextOverflow.ellipsis,
051 |                   ),
052 |                 ),
053 |                 if (onEdit != null)
054 |                   IconButton(
055 |                     tooltip: 'Editar ${note.title}',
056 |                     constraints: const BoxConstraints(
057 |                       minWidth: AppTokens.touchTarget,
058 |                       minHeight: AppTokens.touchTarget,
059 |                     ),
060 |                     onPressed: onEdit,

```

```

061 |         icon: const Icon(Icons.edit_outlined),
062 |     ),
063 |     if (onDelete != null)
064 |     IconButton(
065 |       tooltip: 'Eliminar ${note.title}',
066 |       color: tokens.error,
067 |       constraints: const BoxConstraints(
068 |         minWidth: AppTokens.touchTarget,
069 |         minHeight: AppTokens.touchTarget,
070 |       ),
071 |       onPressed: onDelete,
072 |       icon: const Icon(Icons.delete_outline),
073 |     ),
074 |   ],
075 | ),
076 | const SizedBox(height: AppTokens.space3),
077 | Text(
078 |   note.content,
079 |   style: textTheme.bodyLarge?.copyWith(color: tokens.textMuted),
080 |   maxLines: 5,
081 |   overflow: TextOverflow.ellipsis,
082 | ),
083 | const Spacer(),
084 | const SizedBox(height: AppTokens.space4),
085 | Row(
086 |   children: <Widget>[
087 |     Icon(Icons.calendar_today_outlined, size: AppTokens.space4, color: tokens.textMuted),
088 |     const SizedBox(width: AppTokens.space2),
089 |     Text(_dateLabel(), style: textTheme.labelMedium?.copyWith(color: tokens.textMuted)),
090 |     if (note.authorEmail != null) ...<Widget>[
091 |       const SizedBox(width: AppTokens.space4),
092 |       Expanded(
093 |         child: Text(
094 |           note.authorEmail!,
095 |           textAlign: TextAlign.end,
096 |           overflow: TextOverflow.ellipsis,
097 |           style: textTheme.labelMedium?.copyWith(color: tokens.textMuted),
098 |         ),
099 |       ),
100 |     ],
101 |   ],
102 | ),
103 | ],
104 | ),
105 | ),
106 | );
107 | }
108 | }

```

A.4 StatePanel

Estados: Un enum concentra carga, vacío y error sin duplicar estructura en las pantallas.

Recuperación: actionLabel y onAction permiten crear o reintentar sin acoplar la acción.

Accesibilidad: La región viva anuncia título y mensaje, y cada estado combina icono, texto y color.

```

001 | import 'package:flutter/material.dart';
002 |
003 | import '../design/app_tokens.dart';
004 | import 'app_button.dart';
005 |
006 | enum StatePanelType { loading, empty, error }
007 |
008 | class StatePanel extends StatelessWidget {
009 |   const StatePanel({
010 |     super.key,
011 |     required this.type,
012 |     required this.title,
013 |     required this.message,
014 |     this.actionLabel,
015 |     this.onAction,
016 |   });

```

```

017 |
018 | final StatePanelType type;
019 | final String title;
020 | final String message;
021 | final String? actionLabel;
022 | final VoidCallback? onAction;
023 |
024 | @override
025 | Widget build(BuildContext context) {
026 |     final tokens = AppTokens.of(context);
027 |     final isError = type == StatePanelType.error;
028 |     final icon = type == StatePanelType.empty
029 |         ? Icons.note_add_outlined
030 |         : type == StatePanelType.error
031 |           ? Icons.cloud_off_outlined
032 |           : null;
033 |
034 |     return Semantics(
035 |         liveRegion: true,
036 |         container: true,
037 |         label: '$title. $message',
038 |         child: Container(
039 |             width: double.infinity,
040 |             padding: const EdgeInsets.all(AppTokens.space8),
041 |             decoration: BoxDecoration(
042 |                 color: tokens.surface,
043 |                 borderRadius: BorderRadius.circular(AppTokens.radiusMd),
044 |                 border: Border.all(color: isError ? tokens.error : tokens.outline),
045 |             ),
046 |             child: Column(
047 |                 mainAxisAlignment: MainAxisAlignment.min,
048 |                 children: <Widget>[
049 |                     if (type == StatePanelType.loading)
050 |                         CircularProgressIndicator(
051 |                             color: tokens.primary,
052 |                             semanticsLabel: 'Cargando notas',
053 |                         )
054 |                     else
055 |                         Icon(icon, size: AppTokens.space12, color: isError ? tokens.error : tokens.primary),
056 |                     const SizedBox(height: AppTokens.space4),
057 |                     Text(title, style: Theme.of(context).textTheme.titleLarge, textAlign: TextAlign.center),
058 |                     const SizedBox(height: AppTokens.space2),
059 |                     Text(
060 |                         message,
061 |                         style: Theme.of(context).textTheme.bodyLarge?.copyWith(color: tokens.textMuted),
062 |                         textAlign: TextAlign.center,
063 |                     ),
064 |                     if (actionLabel != null && onAction != null) ...<Widget>[
065 |                         const SizedBox(height: AppTokens.space6),
066 |                         AppButton(
067 |                             label: actionLabel!,
068 |                             onPressed: onAction,
069 |                             variant: isError ? AppButtonVariant.danger : AppButtonVariant.secondary,
070 |                         ),
071 |                     ],
072 |                 ],
073 |             ),
074 |         ),
075 |     );
076 | }
077 | }

```